

Cloud Computing Knowledge Base

Synthetic dataset for RAG experimentation

1. What is Cloud Computing?

Cloud computing is the delivery of computing resources over a network, usually the internet. Instead of purchasing and maintaining physical servers, organizations can rent compute, storage, databases, networking, and other services from cloud providers. Cloud resources can generally be provisioned on demand and scaled according to workload.

2. Virtual Machines

A virtual machine (VM) is a software-defined computer that runs on a physical host through a hypervisor. A VM has virtual CPU, memory, storage, and networking resources. VMs provide strong isolation and are useful when an application needs control over the operating system or runtime environment. A major disadvantage is that each VM includes a full operating system, which consumes additional resources compared with containers.

3. Containers

Containers package an application together with its dependencies while sharing the host operating system kernel. Because containers are lighter than VMs, they can start quickly and allow many workloads to run on the same machine. Docker is a popular container technology, while Kubernetes is commonly used to orchestrate containers across multiple machines.

4. Load Balancing

A load balancer distributes incoming requests across multiple backend servers. Load balancing improves availability and can increase capacity because traffic is spread across several instances. Common strategies include round robin, least connections, and weighted routing. Health checks allow a load balancer to stop sending traffic to an unhealthy server.

5. Auto Scaling

Auto scaling changes the number of running application instances according to demand. For example, an application might run three instances during normal traffic and increase to ten instances during a traffic spike. Scaling can be based on CPU utilization, request rate, queue length, or other application metrics. Scale-out adds instances, while scale-in removes instances.

6. Databases

Cloud applications commonly use relational and NoSQL databases. Relational databases such as MySQL organize data into tables and support SQL and transactions. NoSQL systems can provide flexible schemas and horizontal scalability for particular workloads. Database replication is often used for high availability and read scaling.

7. Object Storage

Object storage stores data as objects inside buckets or containers. It is well suited for images, videos, backups, logs, and large files. Object storage is generally accessed through APIs rather than traditional filesystem operations. Applications often store the file itself in object storage while keeping metadata in a database.

8. Caching

A cache stores frequently accessed data closer to the application so that repeated requests can be served faster. Redis is a commonly used in-memory data store for caching. Caching reduces database load and latency, but applications must handle cache expiration, stale data, and cache invalidation.

9. Message Queues

A message queue allows producers and consumers to communicate asynchronously. A producer places a message into a queue, and a consumer processes it later. Queues can absorb traffic spikes and prevent slow downstream services from blocking the user-facing request. Common concepts include acknowledgments, retries, visibility timeouts, and dead-letter queues.

10. High Availability

High availability means designing a system so that it continues operating despite failures. Techniques include running multiple application instances, distributing workloads across availability zones, using health checks, replicating databases, and avoiding single points of failure. High availability does not necessarily mean zero downtime; it means reducing the probability and impact of service interruption.

Quick Reference

Concept	Primary purpose
Virtual machine	Run an isolated operating system environment
Container	Package and run an application efficiently
Load balancer	Distribute requests across servers
Auto scaling	Adjust capacity based on demand
Database	Persist and query structured or semi-structured data
Object storage	Store large files and blobs
Cache	Reduce latency and backend load
Message queue	Enable asynchronous processing
High availability	Reduce downtime caused by failures